МИНОБРНАУКИ РОССИИ

федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский ядерный университет «МИФИ»

Отчет по сессионному заданию

по дисциплине

«Основы криптографии»

Группа: М26-545

№ студенческого: М265872

Текст задания (вопросы)

1. Опишите трехсторонний протокол распределения ключей между парами участников с использованием третьей стороны.
2. Приведите формальные модели шифров.
3. Приведите принципы построения поточных систем шифрования.
4. Приведите пример режима шифрования, в котором биективность всех шифрующих отображений базового режима не является необходимой для однозначного расшифрования криптограмм. Ответ обоснуйте.
5. Сколько 64-битовых блоков открытого текста искажается при расшифровании криптограммы симметричного блочного шифра, если в шифрованном тексте исказился один бит и используется режим гаммирования с обратной связью по шифртексту, в котором блоки открытого и шифрованного текста имеют длину 8 бит? Ответ обоснуйте.
6. Однонаправленная хэш-функция принимает значения из множества двоичных n-мерных векторов. При каком n случайная пара сообщений составляет коллизию с вероятностью, не большей 10-9? Обязательно должен быть приведен ход решения.

Ответы на поставленные вопросы

№ 1. Опишите трехсторонний протокол распределения ключей между парами участников с использованием третьей стороны:

Рассмотрим трехсторонний протокол распределения ключей между парами участников с использованием третьей стороны основываясь на классическом протоколе Нидхема—Шредера для симметричной криптосистемы.

Участники:

- **A** — инициатор, желающий установить сеансовый ключ с **B**;
- **B** — второй участник;
- **T** — доверенный центр генерации и распределения ключей (KDC), которому доверяют **A** и **B**.

Предполагается, что заранее существуют долговременные секретные ключи:

- K_{AT} — общий ключ A и T;
- K_{BT} — общий ключ B и T.

Сеансовый ключ K_{AB} должен быть выработан T и доставлен A и B.

Основные этапы:

1. **A → T**:

$$A, B, N_A$$

A запрашивает у T сеансовый ключ для связи с B и передаёт одноразовый номер N_A для защиты от повторений.

2. **T → A**:

$$E_{K_{AT}}(N_A, B, K_{AB}, E_{K_{BT}}(K_{AB}, A))$$

T генерирует сеансовый ключ K_{AB}, подтверждает, что запрос действительно от A (по N_A), и отправляет A:

- сеансовый ключ K_{AB};
- «билет» для B — $E_{K_{BT}}(K_{AB}, A)$ зашифрованный на ключе B.

3. **A → B**:

$$E_{K_{BT}}(K_{AB}, A)$$

A пересылает B полученный от T «билет». B расшифровывает его своим долговременным ключом K_{BT} и узнаёт K_{AB} и то, что ключ предназначен для связи с A.

4. **B → A**:

$$E_{K_{AB}}(N_B)$$

B проверяет, что он получил правильный ключ, и подтверждает A свою подлинность с помощью нового одноразового номера N_B.

5. **A → B**:

$$E_{K_{AB}}(N_B - 1)$$

A подтверждает B, что N_B получен и что A действительно владеет K_{AB}.

В итоге между A и B установлен общий сеансовый ключ K_{AB}. T участвует только на этапе выработки и доставки ключа; после завершения протокола A и B используют K_{AB} для защиты дальнейшего обмена. Доверенная третья сторона T знает K_{AB}, но предполагается, что T является доверенным и не будет использовать его во вред.

№ 2. Приведите формальные модели шифров:

Ниже я опишу четыре основные модели.

1. Алгебраическая модель

Шифр задается пятеркой:

$$\Sigma_A = (X, K, Y, E, D),$$

где:

- X — множество открытых текстов;
- K — множество ключей;
- Y — множество шифрованных текстов;
- $E = \{E_k\}_{k \in K}$ — семейство функций зашифрования $E_k : X \to Y$;
- $D = \{D_k\}_{k \in K}$ — семейство функций расшифрования $D_k : Y \to X$.

Основное выполнение условия корректности:

$$D_k(E_k(x)) = x \text{ для всех } x \in X, k \in K.$$

Отсюда следует, что каждое E_k должно быть инъективным: два разных открытых текста на одном ключе не могут дать одинаковую криптограмму, иначе расшифрование было бы неоднозначным.

Назначение: описать устройство шифра и проверить, что он корректно расшифровывается.

2.Вероятностная модель (модель Шеннона):

К алгебраической модели добавляются распределения вероятностей:

$$\Sigma_B = (X, K, Y, E, D, P(X), P(K)),$$

где $P(X)$ задаёт вероятности появления открытых текстов, а $P(K)$ — распределение ключей.

Назначение: оценивать стойкость. Именно в этой модели вводятся:

- совершенная стойкость: $P(x|y)=P(x)$, то есть перехват криптограммы не даёт противнику новой информации об открытом тексте;
- расстояние единственности и другие количественные характеристики.

3. Модель шифрования длинных сообщений (опорный шифр):

Обычно сообщение длиннее одного блока, поэтому его разбивают на части $x_1, x_2, \dots, x_L$ и шифруют базовым (опорным) шифром:

$$E(x_1\, x_2 \dots x_L) = E_{k_1}(x_1)E_{k_2}(x_2) \dots E_{k_L}(x_L).$$

Последовательность ключей $k_1, k_2, \dots$ вырабатывается из основного ключа по некоторому правилу. Если все k_i совпадают, получается шифр простой замены; если ключи меняются — шифр с меняющимся ключом (например, многоалфавитный или поточный).

Назначение: описывать шифрование текстов произвольной длины и сравнивать различные схемы.

4. Автоматная модель:

Шифратор представляется как конечный автомат:

$$A=(S, X, Y, h, f),$$

где:

- S — множество состояний (начальное состояние задаётся ключом);
- X, Y — входной и выходной алфавиты;
- $h : S \times X \to S$ — функция переходов;
- $f : S \times X \to Y$ — функция выходов.

На каждом шаге автомат получает символ открытого текста, выдаёт символ шифртекста и меняет состояние.

Назначение: удобно описывать шифры с «памятью», прежде всего поточные.

Итог: алгебраическая модель описывает устройство шифра, вероятностная — его стойкость, модель длинных сообщений и автоматная — способы шифрования сообщений произвольной длины и шифры с памятью.

№ 3. Приведите принципы построения поточных систем шифрования:

Поточный шифр преобразует открытый текст по битам или небольшим блокам, накладывая на него гамму — псевдослучайную последовательность, выработанную из короткого ключа K и синхропосылки IV.

$$y_i = x_i \oplus \gamma_i,\ x_i = y_i \oplus \gamma_i,$$

Идеал — шифр Вернама (одноразовый блокнот): если гамма истинно случайна, не короче сообщения и используется один раз, шифр совершенно стоек. На практике гамму вырабатывают из короткого ключа, поэтому стойкость зависит от качества генератора.

1. Принципы, обеспечивающие стойкость

- Большой период гаммы.
  Гамма не должна повторяться в пределах шифруемых данных. Иначе при повторном использовании гаммы противник, сложив две криптограммы, получит сумму открытых текстов, что резко упрощает анализ.
- Высокая линейная сложность.
  Если гамма порождается коротким линейным регистром сдвига, то по небольшому её фрагменту алгоритм Берлекэмпа—Мэсси восстановит весь генератор. Поэтому линейная сложность должна быть большой (порядка половины периода).
- Хорошие статистические свойства.
  Гамма должна быть статистически неотличима от случайной: символы и их сочетания встречаются примерно одинаково часто, нет заметных зависимостей. Это защищает от статистического криптоанализа.
- Нелинейное усложнение.
  Линейные регистры сами по себе легко вскрываются, поэтому их выходы пропускают через нелинейную функцию (фильтрующие и комбинирующие генераторы, генераторы с неравномерным движением регистров). Функция должна быть сбалансированной и корреляционно-иммунной, чтобы нельзя было атаковать каждый регистр по отдельности (корреляционная атака).
- Невозможность восстановить ключ по известному фрагменту гаммы.
  Даже если противник знает кусок открытого текста, а значит, и кусок гаммы, он не должен найти ключ или внутреннее состояние генератора быстрее полного перебора. Число ключей должно быть очень большим.
- Одноразовое использование гаммы.
  Нельзя шифровать два сообщения одной и той же гаммой. Для каждого сообщения берётся новая синхропосылка *IV*/*nonce*.
- Отдельный контроль целостности.
  Гаммирование не защищает от подмены: если противник инвертирует бит шифртекста, инвертируется тот же бит открытого текста. Поэтому к поточному шифру добавляют имитовставку (код аутентификации сообщения).

2. Принципы, обеспечивающие эффективность и удобство

- Простота и высокая скорость.
  Генератор строят из простых операций (сдвиги, *XOR*, сложение, табличные замены), чтобы он быстро работал и был дешёв в аппаратной реализации. Это позволяет использовать поточные шифры в связи в реальном времени.

- Отсутствие размножения ошибок.
  В синхронном поточном шифре один искажённый бит шифртекста портит только один бит открытого текста. Это удобно для каналов с помехами.
- Синхронизация.
  Отправитель и получатель должны вырабатывать одинаковую гамму. В синхронных шифрах гамма не зависит от текста, поэтому требуется точная синхронизация; в самосинхронизирующихся она зависит от предыдущих битов шифртекста, что облегчает восстановление синхронизации.

В заключении: принципы 1–7 отвечают за стойкость, 8–10 — за эффективность, помехоустойчивость и удобство эксплуатации.

№ 4. Приведите пример режима шифрования, в котором биективность всех шифрующих отображений базового режима не является необходимой для однозначного расшифрования криптограмм. Ответ обоснуйте:

В качестве примера взят режим гаммирования с обратной связью по шифртексту (*CFB*).

В *CFB* базовое отображение E_K используется только для выработки гаммы, а открытый текст через него не проходит. Поэтому E_K не обязана быть биекцией.

Пусть $y_0 = IV$. Для *i*-го сегмента:

$$\gamma_i = E_k(y_{i-1}),$$
$$y_i = x_i \oplus \gamma_i.$$

При расшифровании:

$$\gamma_i = E_k(y_{i-1}),$$
$$x_i = y_i \oplus \gamma_i.$$

Почему биективность не нужна:

1. Значение $E_k(y_{i-1})$ определено однозначно: у функции на каждый аргумент ровно одно значение. Для этого не требуется инъективность.
2. Получатель знает ключ *k*, синхропосылку *IV* и все блоки шифртекста, которые передаются открыто. Поэтому он подаёт на вход E_k то же самое, что и отправитель, и получает ту же гамму γ_i.
3. Операция $x \mapsto x \oplus \gamma$ при фиксированной γ обратима:
$$(x \oplus \gamma) \oplus \gamma = x.$$

Поэтому x_i однозначно находится как $y_i \oplus \gamma_i$.

Вывод следующий: получается, что обратимость режима обеспечивает операция *XOR* с гаммой, а не функция E_k. Вместо блочного шифра можно было бы взять, например, хэш-функцию, и расшифрование всё равно бы работало.

Так же устроены режим *OFB* ($\gamma_i = E_k(y_{i-1})$) и режим счётчика *CTR* ($\gamma_i = E_k(IV + i)$): в них тоже используется только прямое отображение.

Необходимо отметить, что небиективность не влияет на корректность расшифрования, но может ухудшить свойства гаммы. Например, в *OFB* небиективная функция может дать гамму с более коротким периодом. То есть на однозначность восстановления это не влияет, а на стойкость — может.

№ 5. Сколько 64-битовых блоков открытого текста искажается при расшифровании криптограммы симметричного блочного шифра, если в шифрованном тексте исказился один бит и используется режим гаммирования с обратной связью по шифртексту, в котором блоки открытого и шифрованного текста имеют длину 8 бит? Ответ обоснуйте:

Дано: базовый блочный шифр с длиной блока 64 бита, режим *CFB*, блоки открытого и шифрованного текста по 8 бит. В шифртексте искажён один бит.
Как работает режим. Есть сдвиговый регистр *R* длиной 64 бита, в начале в нём лежит *IV*. На каждом шаге:

$$\gamma_i = MSB_8\left(E_k(R_j)\right),$$
$$y_i = x_i \oplus \gamma_i,$$
$$R_{j+1} = (R_j \ll 8)||y_i.$$

То есть регистр шифруется блочным шифром, из результата берутся старшие 8 бит — это гамма. Гамма складывается с 8-битным блоком открытого текста, получается блок шифртекста. Затем регистр сдвигается на 8 бит, и в освободившееся место записывается этот блок шифртекста. При расшифровании всё так же, только

$$x_j = y_j \oplus \gamma_j,$$

а в регистр записываются принятые блоки шифртекста.
Что происходит при ошибке. Пусть искажён один бит в блоке y_i.

1. Сам блок x_i.
   Гамма γ_i считается по регистру, в котором пока только правильные блоки, поэтому она верная. Тогда
   $$x_i = y_i \oplus \gamma_i.$$
   будет испорчен ровно в одном бите — в том же месте, где ошибка в шифртексте.
2. Следующие блоки.
   Испорченный блок y_i попадает в регистр. Регистр имеет длину 64 бита и вмещает
   $$\frac{64}{8} = 8$$
   блоков по 8 бит. Значит, испорченный блок будет лежать в регистре 8 следующих шагов, пока не «выедет» из него. Всё это время на вход блочного шифра подаётся неправильное значение, и из-за лавинного

эффекта гамма получается совершенно другой. Поэтому блоки $x_{i+1}, \dots, x_{i+8}$ расшифровываются неправильно.

3. Затем.
   Через 8 шагов ошибочный блок полностью выходит из регистра, там остаются только правильные блоки, и расшифрование снова идёт нормально. Это свойство самосинхронизации *CFB*.

Подсчёт:

$$N = 1 + \frac{64}{8} = 1 + 8 = 9.$$

Искажаются 9 блоков открытого текста по 8 бит, то есть

$$9 \cdot 8 = 72 \text{ бита.}$$

В первом блоке испорчен один бит, в остальных восьми — весь блок (точнее, все биты соответствующего 8-битного сегмента могут быть искажены из-за неверной гаммы).

Если смотреть на 64-битные блоки открытого текста: эти 72 подряд идущих бита всегда попадают в два соседних 64-битных блока. Они не могут уместиться в одном блоке, так как 72 > 64, но и не могут затронуть три блока, поскольку для трёх блоков при таком количестве сегментов нет места.

Краевой случай: если ошибка случилась близко к концу сообщения, испорченных блоков будет меньше, просто потому что дальше текста нет.

Ответ следующий: искажаются 9 сегментов по 8 бит (72 бита), что соответствует двум 64-битным блокам открытого текста.

№ 6. Однонаправленная хэш-функция принимает значения из множества двоичных n-мерных векторов. При каком n случайная пара сообщений составляет коллизию с вероятностью, не большей 10-9? Обязательно должен быть приведен ход решения:

Дано: хэш-функция принимает значения из множества двоичных векторов длины n, то есть всего возможно

$$N = 2^n$$

разных значений. Нужно найти n, при котором случайная пара сообщений даёт коллизию с вероятностью не больше 10^{-9}.

Модель. Используем модель парадокса дней рождения: считаем, что хэш-значения разных сообщений случайны, независимы и равномерно распределены по N значениям.

Для k сообщений вероятность того, что все хэши разные:

$$P_{\text{нет}}(k) = \left(1 - \frac{1}{N}\right)\left(1 - \frac{2}{N}\right) \dots \left(1 - \frac{k-1}{N}\right).$$

Вероятность хотя бы одной коллизии:

$$\text{P(k)} = 1 - P_{\text{нет}}(k) \approx 1 - e^{-\frac{k(k-1)}{2N}}.$$

Решение. У нас пара сообщений, значит k=2. В произведении остаётся только одна скобка, и приближение не нужно:

$$P(2) = \left(1 - \frac{1}{N}\right) = \frac{1}{N} = 2^{-n}.$$

Это понятно и без формулы: хэш первого сообщения какой-то, а хэш второго совпадёт с ним с вероятностью 1 из 2^n.

Требуется:

$$2^{-n} \leq 10^{-9}.$$

Отсюда:

$$2^n \geq 10^9.$$

Логарифмируем по основанию 2:

$$n \geq log_2(10^9) = 9log_2 10.$$

Считаем:

$$log_2 10 = \frac{1}{lg_2} \approx \frac{1}{0{,}30103} \approx 3{,}3219.$$

Тогда:

$$n \geq 9 \cdot 3{,}3219 \approx 29{,}9.$$

Так как n целое:

$$n \geq 30.$$

Проверка:

- при $n = 29$: $2^{29} = 536\,870\,912 < 10^9$, вероятность $\approx 1{,}86 \cdot 10^{-9}$— больше 10^{-9}, не подходит;
- при $n = 30$: $2^{30} = 1073741824 > 10^9$, вероятность $\approx 9{,}31 \cdot 10^{-10}$ — меньше 10^{-9}, подходит.

Ответ: $n \geq 30$, то есть хэш-значение должно иметь длину не меньше 30 бит.

Замечание. Это верно только для одной случайной пары. Если противник может перебирать много сообщений, вероятность коллизии растёт примерно как

$$\frac{k^2}{2^{n+1}},$$

и коллизию можно найти примерно за $2^{n/2}$ попыток. Для $n = 30$ это всего около $22^{15} \approx 33$ тысяч сообщений. Поэтому у настоящих хэш-функций длина значения намного больше, например 256 бит.